Escape Dungeon Q-Learning Game

1. Introduction

The game designed for this assignment is "Dungeon Escape", which is a grid-based maze escape game. It requires the Q-learning agent to navigate through the dungeon grid, avoiding traps and random sliding tiles, and finally reaching the exit.
The core objective of the game is to have the agent move from the designated starting point to the destination while maintaining the highest score; the game ends in two scenarios: the agent steps on a trap (failure) or reaches the exit (success).

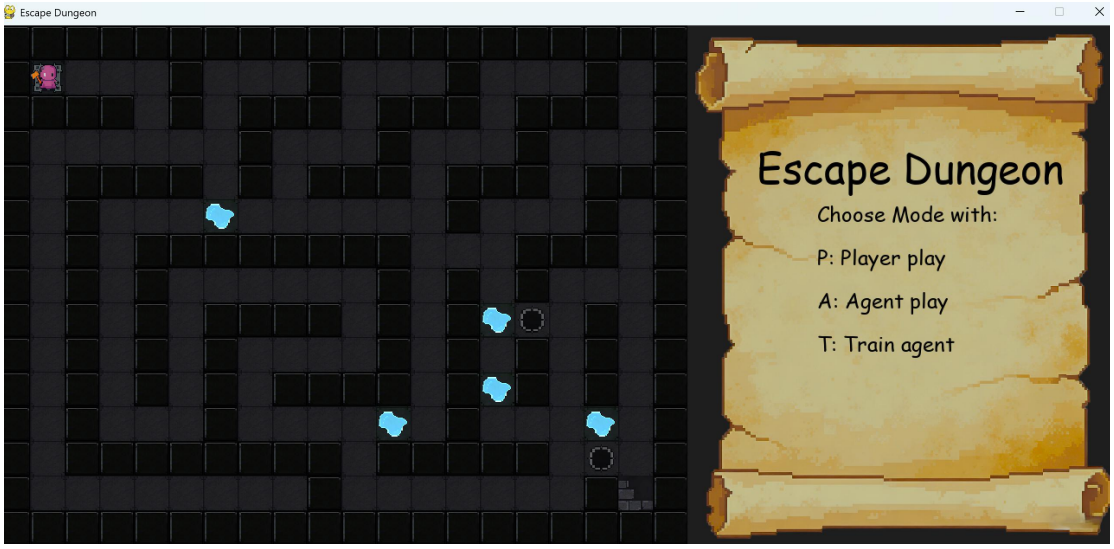

Fig 1.1 game interface

2. Game Design

2.1 rules of the game

The player's initial score is 100 points. One point is deducted for each step taken, 50 points are deducted for stepping on a trap, and the game is immediately declared a failure and ends. 50 points are added upon reaching the exit, and the game is immediately declared a success and ends.
The initial score of the agent is 100 points. For each step of movement, 1 point is deducted. This is to motivate the agent to find an efficient path.

- If the agent steps into a trap, 100 points will be deducted and the game will immediately determine failure and end.
- If the agent reaches the exit, 500 points will be added and the game will immediately determine success and end.

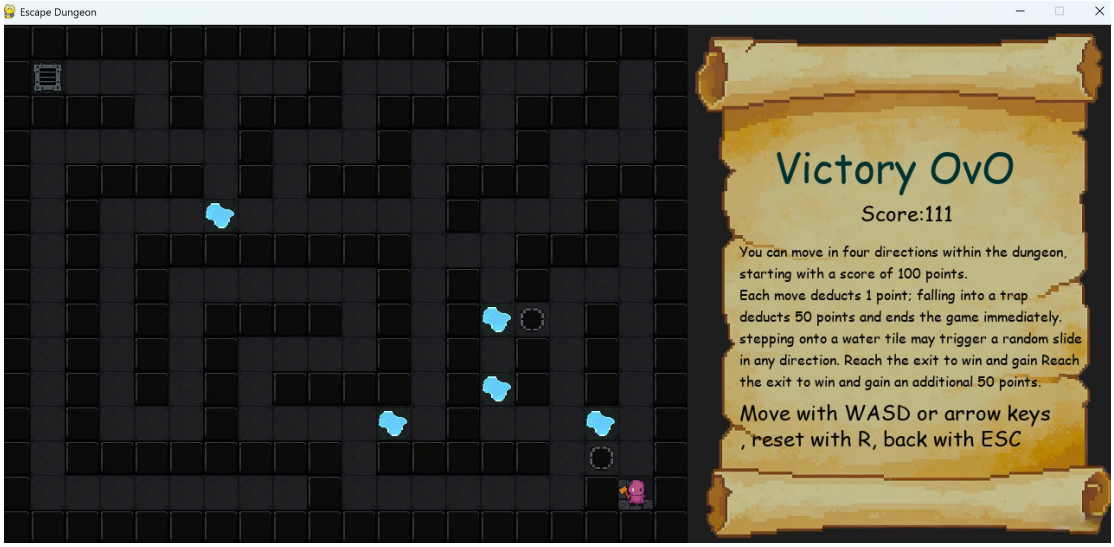

Fig 2.1 game victory interface

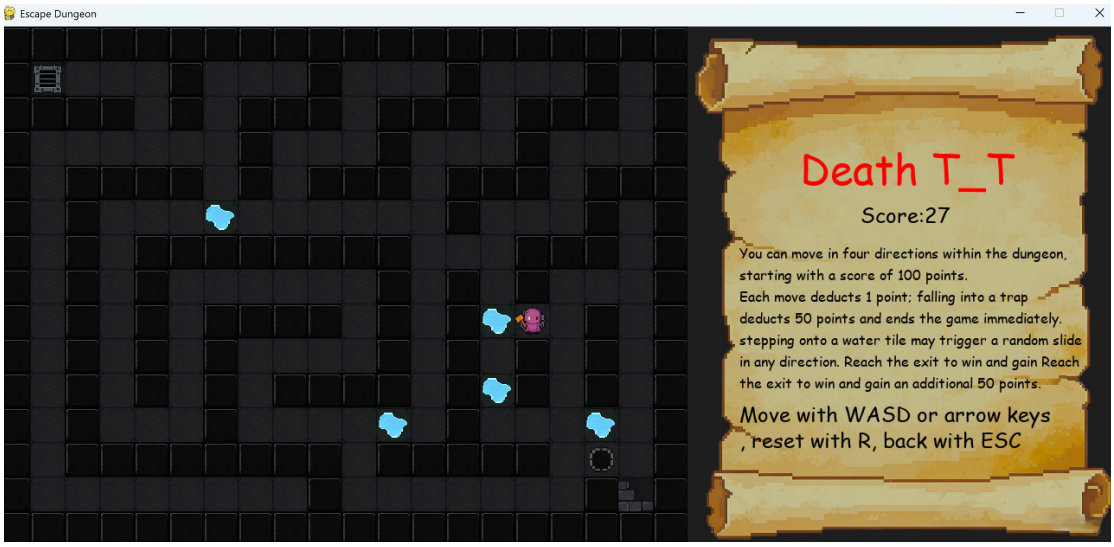

Fig 2.2 game failure interface

The sliding tiles have a 50% probability of forcing the agent to move in a randomly legal direction, thereby increasing the uncertainty of the environment.

The agent can only move in four directions (up, down, left, right) and cannot pass through walls. The P key is for the player's gameplay mode, the T key is for the agent's training mode (default 2000 rounds), the A key is for the demonstration mode of the agent's training results, during which the agent directly uses the decayed ε - value after training without any additional manual settings. The R key can be used to reset the game, and the ESC key can be used to return to the mode selection interface for manual testing or re-training.

2.2 Class design of the game

This game is developed based on Python and Pygame using a modular

object-oriented architecture. The core classes include:

Additional engineering logic: Before the program runs, it will automatically detect and clear the out/training_log.txt training log file to avoid interference from historical data; if the out folder does not exist, it will be automatically created to solve the problem of error in log file writing path.

config: A centralized configuration class for all game parameters (window size, map layout, Q-learning hyperparameters, reward values, color values).

util: A utility class that implements coordinate conversion, random direction selection, probability judgment, state/action indexing, etc.

GameMap: A map management class that parses the grid layout, identifies key positions (start point / end point / traps / slippery tiles / walls), and returns legal movement directions for any state.

Player: An agent/player class that handles movement logic, score tracking, position reset, including random movement logic triggered by slippery tiles.

ui: A UI rendering class that draws the map, agent, score, game state (win / lose), and game instructions, with the maze area (800×600) and information panel area (500×600) displayed separately.

QTable: A Q-table storage class that initializes a zero matrix (300 states × 5 actions), and provides methods for getting / setting Q values.

QLearningAgent: The core Q-learning agent class that implements ε-greedy action selection, Q-value update, and ε-value decay logic.

Game: The main game class that handles event loop, human/agent game flow, training logic, and result log recording.

Map layout diagram: The map uses digital encoding to mark grid cells (0 = empty space, 1 = wall, 2 = trap, 3 = slippery tile, 4 = start point, 5 = end point). The start point is at (1,1) (the first row and first column), and the end point is at (18,13) (the 13th row and 18th column). Traps and slippery tiles are strategically placed in the middle and later sections of the maze to increase difficulty, and walls form complex paths that require the agent to learn non-simple navigation strategies.

The QTable class initializes a zero matrix with 300 states and 5 actions. The action dimension includes four valid movements (up, down, left, right) and one empty action. There is a mapping logic between the adapted coordinates and the action indices.

2.3 UI design

The user interface built based on Pygame fully supports the two-way interaction between the Q-learning agent and the game environment:

Agent to Environment: The agent selects an action through the Q-learning algorithm, and this action is passed to the Player class for movement execution; the environment (GameMap) verifies the legality of the action and returns the new state.

Environment to Agent: The environment sends immediate rewards, the next state, and the game over flag to the agent. The agent uses these values to update the Q-table in real time.

Human interaction: This interface also supports manual gameplay (using WASD / arrow keys for movement, and the R key for reset). Users can manually test the difficulty of the maze and compare the performance with that of the intelligent agent.

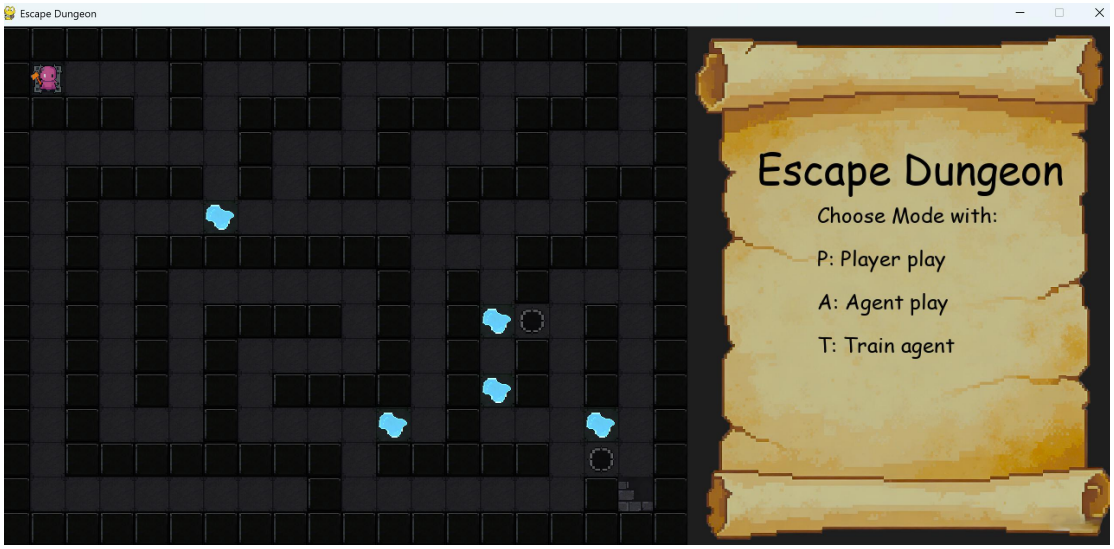

Fig 2.3 game UI design

3. Q-Learning Implementation

3.1 Q-learning algorithm

The core Q-value update formula is:

$$Q(s,a) = Q(s,a) + \alpha \times \left[r + \gamma \times \max_{a'} Q(s',a') - Q(s,a) \right]$$

Initialize a fully zero Q-table (300 states $\times$ 5 actions) using the Q_Table class. Then, in each training round, the agent starts from the initial state and selects an action through the ε-greedy strategy. The agent executes this action, and the environment returns the next state s', the immediate reward r, and the game end flag. Update the Q value of the current (s,a) pair using the time difference formula.

If the game ends (winning / losing), the agent resets to the starting point and the ε-value decays; repeat the above process to enter the next training round.

```python
# Q表容器类
class QTable:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.Q_value = np.zeros((self.state_size, self.action_size))

    def get_q_value(self, state, action):
        return self.Q_value[util.coordinates_to_index(state), util.action_to_index(action)]

    def set_q_value(self, state, action, value):
        self.Q_value[util.coordinates_to_index(state), util.action_to_index(action)] = value
```

Fig 3.1 Q-table

3.2 Strategy learning for maximizing cumulative rewards

The agent learns through training a set of deterministic optimal policies (mapping from states to actions) to maximize the cumulative discounted rewards over the long term. The total number of training rounds is set to 2000 (configurable). The core features of policy learning are as follows:

Cumulative Reward Tracking: The total reward for each training round is the sum of the step reward and the victory/defeat reward. This value is simultaneously recorded in the console and the training log file (out/training_log.txt).

Policy Convergence: As the training progresses, the success rate of the agent significantly increases. The cumulative reward for a single round rises from a negative value (during the exploration phase in the early training stage) to a positive value (during the exploitation phase in the later training stage), indicating that the policy converges to the optimal behavior.

Shortest Path Learning: The negative movement reward (-1) encourages the agent to learn the shortest path to the exit. The number of steps required to successfully escape gradually decreases as the training progresses.

Failure Avoidance: The high negative failure reward (-100) causes the agent to form a conditioned reflex, avoiding trap units and the slippery tiles near them, minimizing the number of failure rounds in the later training stage.

3.3 ε -greedy strategy, learning rate and discount factor

To balance exploration (searching for new paths) and exploitation (utilizing the known optimal paths), this project has optimized three core hyperparameters effectively and all parameter values are centralized in the config class for easy adjustment:

ε -greedy exploration:

Initial ε value: 0.8 (80% probability of random exploration, 20% probability of using the optimal action).

Epsilon decay rate: 0.99 (After each training round, the ε value is multiplied by 0.99).

Minimum ε value: 0.01 (Stop the decay and retain the minimum level of exploration).

The demonstration mode does not set an additional ε value. It directly uses the final decay value obtained after training to ensure that the demonstration strategy is completely consistent with the training convergence strategy.

This decaying ε value strategy ensures that the agent fully explores the environment in the early stages of training, utilizes the learned optimal strategy in the later stages of training, and avoids getting stuck in a local optimal solution.

Learning rate (α): 0.3. The moderate learning rate strikes a balance between the agent's ability to update the Q values based on new experiences and its capacity to retain past learning achievements. It prevents useful Q values from being overwritten and also avoids overly slow convergence speed.

Discount factor (γ): 0.9. A higher discount factor makes the agent place greater emphasis on future rewards (such as a +500 victory reward) rather than immediate step rewards (-1), aligning the agent's behavior with the long-term escape goal rather than simply avoiding short-term negative rewards.

```
# QLearningAgent参数
Q_GAMMA = 0.9
Q_EPSILON = 0.8
Q_EPSILON_DECAY = 0.99
Q_EPSILON_MIN = 0.01
Q_LEARNING_RATE = 0.3
Q_WIN_REWARD = 500
Q_LOSS_REWARD = -100
Q_MOVE_REWARD = -1
```

Fig 3.1 parameter setting

4. Challenges and Solutions

Challenge 1: Map Design

Problem: The map design should have both playability and be suitable for the intelligence agent to learn gradually. If it is too simple, the agent will master it in one second and there will be no learning value. If it is too difficult, with too many traps and complex paths, it will result in a low win rate for the agent.

Solution: Adopt a map structure that is easier at the beginning and becomes more difficult later: Set simple straight paths near the starting point to reduce the initial learning difficulty; gradually increase traps and sliding bricks in the later part of the maze to enhance the playability of the game and allow the agent to learn strategies step by step.

Challenge 2: Uncertainty of the sliding tiles leads to instability in training

Problem: The 50% random sliding probability of the sliding tiles introduces randomness in the environment, resulting in inconsistent state transitions and slow policy convergence (the agent has difficulty learning predictable paths).

Solution: Modify the logic of the sliding tiles, allowing only random sliding in legal directions (excluding walls) to avoid the agent being forced into invalid states. Increase the penalty value of the failure reward from -50 to -100, strongly preventing the agent from moving towards sliding tiles near traps. Increase the number of training rounds from 300 to 2000, allowing the agent to gain more experience and learn the regularities of random state transitions.

Challenge 3: Parameter tuning of the ε -greedy strategy to achieve a balance between exploration and exploitation

Problem: When the initial ε value is set to 0.9, the exploration behavior is excessive (random movement is too frequent, and the win rate increases slowly); when the initial ε value is too low (0.5), the exploration behavior is insufficient (the agent gets stuck in a local optimal solution and cannot find the exit).

Solution: Set the initial ε value to 0.8 (to balance exploration and exploitation), and combine it with a low decay rate (0.99) to ensure sufficient exploration in the early training stage. Set the minimum ε value (0.01), and retain the minimum level of exploration in the later training stage to prevent the agent's behavior from being too deterministic and unable to adapt to minor changes in the environment.

5. Conclusion

This assignment successfully designs and implements a grid-based Q-learning game (Escape Dungeon) using Python and Pygame, meeting all project requirements for game design, Q-learning implementation, UI interaction, and documentation.

The agent completed 2000 rounds of training using the hyperparameters defined in the config class. The core evaluation results are as follows:

Win rate: In the initial training phase (up to 100 rounds), the win rate was 32% (the agent randomly explored and frequently stepped into traps / failed to move). After the 400th round: the average win rate over 1600 consecutive rounds reached 96.3%, and in the last 100 rounds (1901-2000), there were only 7 failures, all due to 50% random sliding of the tiles, unrelated to the agent's strategy.

Cumulative Reward: The average cumulative reward per round increased from -150 points in the early stage of training to +450 points in the later stage of training. A successful escape round can earn a cumulative reward of approximately 480 points (500 points for a victory bonus minus the movement reward of about 20 steps for the shortest path).

Step length efficiency: The average number of steps taken to successfully escape decreased from approximately 80 steps in the early stages of training to approximately 20 steps in the later stages, indicating that the agent has learned the shortest path to the exit.

Robustness: In the later stage of training, the agent was able to successfully avoid all trap units and learned the navigation strategy in the slip tile area (with a 50% slip probability) by avoiding the tiles near the traps, demonstrating its adaptability to environmental uncertainties.

The project validates the core concepts of reinforcement learning and Q-learning, providing practical experience in algorithm implementation, game design, and hyperparameter tuning.